SET F

OUTPUT, INPUT, OBJECT, POLYMORPHISM, ENCAPSULATION

1️⃣ OUTPUT (Displaying Result on Screen)

🔹 **What is Output?**

Output means showing information on the computer screen.

In C++, we use **output** to display:

- Text
- Numbers
- Results of calculations

🔹 **Output Statement in C++**

In C++, output is done using:

cout

(Pronounced as **"see-out"**)

It is used with:

<<

(called **insertion operator**)

🔹 **Example:**

```
cout << "Hello World";
```

🔹 **Output of the Program:**

```
Hello World
```

🔹 **Printing Numbers:**

cout << 10;

🔹 **Printing Multiple Items:**

cout << "Age is " << 20;

🔹 **Important Points:**

✓ cout is used to show output
✓ << sends data to the screen
✓ Output helps us see program results

2️⃣ INPUT (Taking Data from User)

🔹 **What is Input?**

Input means taking data **from the user** using the keyboard.

Examples of input:

- Name
- Age
- Marks

🔹 **Input Statement in C++**

In C++, input is taken using:

cin

(Pronounced as **"see-in"**)

It is used with:

>>

(called **extraction operator**)

🔹 **Example:**

int age;

cin >> age;

This means:

- Ask the user to enter age
- Store it in the variable age

🔹 **Complete Input-Output Example:**

int number;

cin >> number;

cout << number;

🔹 **Important Points:**

✓ cin takes input from user
✓ >> stores data into variable
✓ Input makes programs interactive

3️⃣ POLYMORPHISM

🔹 **Meaning of Polymorphism**

The word **Polymorphism** means:

One name, many forms

🔹 **Simple Explanation:**

Polymorphism allows **one function or object** to behave in **different ways**.

🔹 **Real-Life Example:**

A person can be:

- A teacher in school
- A parent at home
- A customer in a shop

➞ Same person, **different roles**

🔹 **Polymorphism in C++:**

- Same function name
- Different behavior

Example idea (no need to remember code deeply):

add(int a, int b);

add(double a, double b);

🔹 **Why Polymorphism is Useful?**

✓ Makes code flexible
✓ Reduces repetition
✓ Easy to manage programs

🔹 **Key Points:**

✓ One name, many forms
✓ Improves program design
✓ Important concept of OOP

4️⃣ **ENCAPSULATION**

🔹 **Meaning of Encapsulation**

Encapsulation means:

Wrapping data and functions together

🔹 **Simple Explanation:**

Encapsulation keeps **data safe** by hiding it from direct access.

🔹 **Real-Life Example:**

A **capsule medicine**:

- Medicine is inside
- You cannot see or touch it directly

Same in C++:

- Data is hidden
- Access is controlled

🔹 **Encapsulation in C++:**

- Data (variables) are kept **private**
- Functions are used to access data safely

🔹 **Why Encapsulation is Important?**

✓ Protects data
✓ Prevents misuse
✓ Makes code secure

🔹 **Key Points:**

✓ Data hiding concept
✓ Improves security
✓ Part of Object-Oriented Programming

5️⃣ OBJECT

🔹 What is an Object?

An **object** is a **real-world entity**.

It has:

- **Properties** (data)
- **Actions** (functions)

🔹 Real-Life Example:

Car is an object:

- Color
- Model
- Speed

Actions:

- Drive
- Stop

🔹 Object in C++:

An object is created from a **class**.

Example:

Car myCar;

Here:

- Car → class

- myCar → object

🔹 **Why Objects are Important?**

✓ Represent real-life things
✓ Make programs easy to understand
✓ Used in Object-Oriented Programming

🔹 **Key Points:**

✓ Object is an instance of a class
✓ Contains data and functions
✓ Helps in real-world programming

📌 **EXAM QUICK REVISION**

Topic	One-Line Definition
Output	Display data on screen
Input	Take data from user
Polymorphism	One name, many forms
Encapsulation	Wrapping data and functions
Object	Real-world entity in program

Encapsulation & Polymorphism (Detailed)

1️⃣ **Encapsulation**

🔹 **Definition**

Encapsulation means hiding data and allowing access to it only through methods.
It combines **data (variables)** and **functions** into one unit called a **class**.

👉 In short: **Data hiding + controlled access**

🔹 **Real-life example** 🧠

ATM Machine

- You cannot directly touch the cash 💵
- You use buttons (functions)
- PIN and balance are hidden

✓ Data is protected
✓ Access is controlled

🔹 **How Encapsulation works in C++**

- Data members → private
- Functions → public
- Access using **getters and setters**

🔹 **Example (Encapsulation in C++)**

```
#include <iostream>

using namespace std;


class Student {

private:

   int marks;   // hidden data


public:

   void setMarks(int m) {

      marks = m;
```

```
    }

    int getMarks() {
        return marks;
    }
};

int main() {
    Student s;
    s.setMarks(85);
    cout << s.getMarks();
}
```

📝 **Explanation**:

- marks cannot be accessed directly
- It is protected using private
- Accessed safely using functions

🔹 **Why Encapsulation is important**

✓ Protects data
✓ Improves security
✓ Prevents misuse
✓ Easy to manage code

🔹 **One-line summary**

Encapsulation hides data and provides controlled access through methods.

2️⃣ Polymorphism

🔹 Definition

Polymorphism means "many forms".
It allows **one function name** to perform **different tasks**.

🔹 Real-life example 🧠

Person roles

- A man is a **father** at home 👨‍👧
- A **teacher** in school 👩‍🏫
- A **customer** in a shop 🛒

Same person, **different behavior** → Polymorphism

🔹 Types of Polymorphism in C++

🔸 1. Compile-time Polymorphism (Function Overloading)

✓ Same function name
✓ Different parameters

```
#include <iostream>

using namespace std;


int add(int a, int b) {

   return a + b;

}


int add(int a, int b, int c) {

   return a + b + c;
```

```
}

int main() {
    cout << add(2, 3) << endl;
    cout << add(1, 2, 3);
}
```

📝 **Explanation**:

- add() works in different ways
- Compiler decides which one to use

🔸 **2. Runtime Polymorphism (Function Overriding)**

✓ Same function
✓ Different behavior
✓ Uses virtual keyword

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() {
        cout << "Animal sound" << endl;
    }
};

class Cat : public Animal {
public:
```

```
    void sound() {
        cout << "Cat meows" << endl;
    }
};

int main() {
    Animal* a;
    Cat c;
    a = &c;
    a->sound();
}
```

📝 **Explanation**:

- Same function sound()
- Output depends on object type

🔹 **Why Polymorphism is important**

✓ Code reuse
✓ Flexibility
✓ Easy expansion
✓ Cleaner design

🔹 **One-line summary**

Polymorphism allows the same function to behave differently.

🔁 **Encapsulation vs Polymorphism (Easy Table)**

Feature	Encapsulation	Polymorphism

Feature	Encapsulation	Polymorphism
Meaning	Data hiding	Many forms
Focus	Security	Flexibility
Example	ATM machine	Same person, many roles
Main benefit	Protects data	Reusable behavior